

Contents

1 Datasets for IDS Research	1
1.1 Comprehensive Dataset Information and Preprocessing Pipeline	1
1.2 Table of Contents	2
1.3 1. Dataset Overview	2
1.3.1 1.1 Summary Table	2
1.3.2 1.2 Dataset Selection Rationale	2
1.4 2. NSL-KDD Dataset	3
1.4.1 2.1 Overview	3
1.4.2 2.2 Attack Categories	3
1.4.3 2.3 Feature Description	4
1.4.4 2.4 Preprocessing Steps	4
1.5 3. CICIDS2017/2018 Dataset	6
1.5.1 3.1 Overview	6
1.5.2 3.2 Attack Types by Day	6
1.5.3 3.3 Feature Categories	6
1.5.4 3.4 Preprocessing Steps	7
1.6 5. IoT-23 Dataset	11
1.6.1 5.1 Overview	11
1.6.2 5.2 Malware Types	11
1.6.3 5.3 Feature Extraction	11
1.7 6. 5G/Custom Dataset Collection	12
1.7.1 6.1 Collection Plan	12
1.7.2 6.2 Data Collection Tools	13
1.8 7. Data Preprocessing Pipeline	13
1.8.1 7.1 Complete Pipeline	13
1.9 8. Train/Validation/Test Splits	16
1.9.1 8.1 Split Strategy	16
1.10 9. Class Imbalance Handling	16
1.10.1 9.1 SMOTE (Synthetic Minority Over-sampling)	16
1.10.2 9.2 Class Weights	17
1.11 10. Data Augmentation Strategies	17
1.11.1 10.1 Gaussian Noise Injection	17
1.11.2 10.2 Feature Masking	17
1.11.3 10.3 Time-Series Augmentation	17
1.12 Conclusion	18

1 Datasets for IDS Research

1.1 Comprehensive Dataset Information and Preprocessing Pipeline

Dataset Specifications, Characteristics, and Usage Guidelines

Last Updated: December 22, 2025

1.2 Table of Contents

1. Dataset Overview
2. NSL-KDD Dataset
3. CICIDS2017/2018 Dataset
4. UNSW-NB15 Dataset
5. IoT-23 Dataset
6. 5G/Custom Dataset Collection
7. Data Preprocessing Pipeline
8. Train/Validation/Test Splits
9. Class Imbalance Handling
10. Data Augmentation Strategies

1.3 1. Dataset Overview

1.3.1 1.1 Summary Table

Dataset	Year	Samples	Features	Attack Types	Protocol Coverage	Source
NSL-KDD	2009	148,517	41	4 categories, 37 types	TCP, UDP, ICMP	Refined KDD'99
CICIDS2017	2017	2.8M flows	80	14 types	HTTP, HTTPS, FTP, SSH, etc.	Canadian Inst. Cyber-security
UNSW-NB15	2015	2.54M	49	9 categories	TCP, UDP, ICMP, etc.	UNSW Canberra
IoT-23	2020	325M pkts	Variable	20+ IoT malware	IoT protocols (MQTT, CoAP)	Avast AIC
5G Custom	2025+	TBD	TBD	NGN-specific	5G NR, Slicing	Testbed collection

1.3.2 1.2 Dataset Selection Rationale

NSL-KDD: - Widely used benchmark for comparison with existing work - Balanced classes (no excessive duplicate records like KDD'99) - Established baseline for algorithm validation

CICIDS2017: - Modern attack types (DDoS, web attacks, botnet) - Realistic network traffic (captured from real infrastructure) - Labeled flows with 80+ features for deep learning

UNSW-NB15: - Hybrid synthetic and real attacks - Diverse attack categories (9 types) - Contemporary attack techniques (2015)

IoT-23: - IoT-specific malware (Mirai, Torii, etc.) - Real IoT device captures - Essential for IoT security research

5G Custom: - Addresses gap in 5G-specific attack datasets - Network slicing attacks - Core network vulnerabilities

1.4 2. NSL-KDD Dataset

1.4.1 2.1 Overview

NSL-KDD is a refined version of the KDD Cup 1999 dataset, addressing issues such as duplicate records and imbalanced classes.

Statistics: - Training set: 125,973 records - Test set: 22,544 records - Features: 41 (38 numerical, 3 categorical) - Classes: Normal + 4 attack categories

1.4.2 2.2 Attack Categories

Category	Description	Types	% in Train	% in Test
DoS	Denial of Service	back, land, neptune, pod, smurf, teardrop	45.93%	28.67%
Probe	Surveillance/Scanning	isweep, nmap, portsweep, satan	11.66%	12.34%
R2L	Remote to Local	ftp_write, guess_passwd, imap, multihop, phf, spy, warez-client, warez-master	0.83%	23.63%
U2R	User to Root	buffer_overflow, loadmodule, perl, rootkit	0.04%	0.71%
Normal	Legitimate traffic	-	53.54%	34.65%

1.4.3 2.3 Feature Description

Basic Features (9): 1. duration: Connection duration (seconds) 2. protocol_type: TCP, UDP, ICMP 3. service: HTTP, FTP, SMTP, etc. (70 values) 4. flag: Connection status (SF, SO, REJ, etc.) 5. src_bytes: Bytes from source to destination 6. dst_bytes: Bytes from destination to source 7. land: 1 if connection from/to same host/port 8. wrong_fragment: Number of wrong fragments 9. urgent: Number of urgent packets

Content Features (13): 10. hot: Number of “hot” indicators 11. num_failed_logins: Number of failed login attempts 12. logged_in: 1 if successfully logged in 13. num_compromised: Number of compromised conditions 14. root_shell: 1 if root shell obtained 15. su_attempted: 1 if su command attempted 16. num_root: Number of root accesses 17. num_file_creations: Number of file creation operations 18. num_shells: Number of shell prompts 19. num_access_files: Number of operations on access control files 20. num_outbound_cmds: Number of outbound commands (FTP) 21. is_host_login: 1 if login belongs to host list 22. is_guest_login: 1 if guest login

Traffic Features (9): 23-31. count, srv_count, error_rate, srv_error_rate, error_rate, srv_error_rate, same_srv_rate, diff_srv_rate, srv_diff_host_rate

Time-based Traffic Features (9): 32-41. dst_host_count, dst_host_srv_count, dst_host_same_srv_rate, dst_host_diff_srv_rate, dst_host_same_src_port_rate, dst_host_srv_diff_host_rate, dst_host_error_rate, dst_host_srv_error_rate, dst_host_error_rate, dst_host_srv_error_rate

1.4.4 2.4 Preprocessing Steps

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler

def preprocess_nslkdd(file_path):
    """
    Preprocess NSL-KDD dataset
    """

    # Column names
    columns = ['duration', 'protocol_type', 'service', 'flag', 'src_bytes',
               'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot',
               'num_failed_logins', 'logged_in', 'num_compromised',
               'root_shell', 'su_attempted', 'num_root', 'num_file_creations',
               'num_shells', 'num_access_files', 'num_outbound_cmds',
               'is_host_login', 'is_guest_login', 'count', 'srv_count',
               'error_rate', 'srv_error_rate', 'rerror_rate',
               'srv_rerror_rate', 'same_srv_rate', 'diff_srv_rate',
               'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count',
               'dst_host_same_srv_rate', 'dst_host_diff_srv_rate',
               'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate',
               'dst_host_error_rate', 'dst_host_srv_error_rate',
```

```

        'dst_host_rerror_rate', 'dst_host_srv_rerror_rate', 'label', 'difficulty'

# Load data
df = pd.read_csv(file_path, names=columns)

# Map attack types to categories
attack_mapping = {
    'normal': 'Normal',
    'back': 'DoS', 'land': 'DoS', 'neptune': 'DoS', 'pod': 'DoS',
    'smurf': 'DoS', 'teardrop': 'DoS', 'apache2': 'DoS', 'udpstorm': 'DoS',
    'ipsweep': 'Probe', 'nmap': 'Probe', 'portsweep': 'Probe', 'satan': 'Probe',
    'mscan': 'Probe', 'saint': 'Probe',
    'ftp_write': 'R2L', 'guess_passwd': 'R2L', 'imap': 'R2L',
    'multihop': 'R2L', 'phf': 'R2L', 'spy': 'R2L', 'warezclient': 'R2L',
    'warezmaster': 'R2L', 'sendmail': 'R2L', 'named': 'R2L',
    'snmpgetattack': 'R2L', 'snmpguess': 'R2L', 'xlock': 'R2L', 'xsnoop': 'R2L',
    'buffer_overflow': 'U2R', 'loadmodule': 'U2R', 'perl': 'U2R',
    'rootkit': 'U2R', 'httptunnel': 'U2R', 'ps': 'U2R', 'sqlattack': 'U2R'
}

df['attack_category'] = df['label'].str.strip().map(attack_mapping)

# Encode categorical features
categorical_cols = ['protocol_type', 'service', 'flag']
label_encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Separate features and labels
X = df.drop(['label', 'attack_category', 'difficulty'], axis=1)
y = df['attack_category']

# Normalize numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

return X_scaled, y, label_encoders, scaler

```

2.5 Usage Guidelines

```

**Binary Classification (Normal vs. Attack):**
```python
y_binary = y.map({'Normal': 0, 'DoS': 1, 'Probe': 1, 'R2L': 1, 'U2R': 1})

```

## Multi-class Classification (5 classes):

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_multiclass = le.fit_transform(y) # 0: Normal, 1: DoS, 2: Probe, 3: R2L, 4: U2R
```

---

## 1.5 3. CICIDS2017/2018 Dataset

### 1.5.1 3.1 Overview

The Canadian Institute for Cybersecurity Intrusion Detection System (CICIDS2017) dataset contains labeled network flows with modern attack types.

**Statistics:** - Total flows: ~2.8 million - Features: 80 (78 numerical, 2 categorical) - Duration: 5 days (Monday-Friday) - Attacks: 14 types across various categories

### 1.5.2 3.2 Attack Types by Day

Day	Date	Traffic	Attacks
Monday	7/3/2017	Normal	None (baseline)
Tuesday	7/4/2017	Brute Force, XSS	FTP-Patator, SSH-Patator, XSS
Wednesday	7/5/2017	DoS/DDoS	DoS GoldenEye, DoS Hulk, DoS Slowhttptest, DoS Slowloris, Heartbleed
Thursday	7/6/2017	Web Attacks, Infiltration	Web Attack (Brute Force, XSS, SQL Injection), Infiltration
Friday	7/7/2017	Botnet, Port Scan, DDoS	Botnet ARES, Port Scan, DDoS

### 1.5.3 3.3 Feature Categories

#### Flow-based Features (80 total):

##### 1. Flow Identifiers:

- Flow ID, Source IP, Source Port, Destination IP, Destination Port, Protocol, Timestamp

##### 2. Duration:

- Flow Duration

##### 3. Packet Counts:

- Total Fwd Packets, Total Backward Packets

##### 4. Byte Counts:

- Total Length of Fwd Packets, Total Length of Bwd Packets

#### 5. **Packet Length Statistics:**

- Fwd/Bwd Packet Length Max, Mean, Min, Std

#### 6. **Inter-Arrival Time (IAT):**

- Flow IAT Mean, Std, Max, Min
- Fwd IAT Total, Mean, Std, Max, Min
- Bwd IAT Total, Mean, Std, Max, Min

#### 7. **Flags:**

- PSH Flag Count, URG Flag Count, FIN Flag Count, SYN Flag Count, RST Flag Count, ACK Flag Count

#### 8. **Header Lengths:**

- Fwd/Bwd Header Length

#### 9. **Packets/Second:**

- Flow Packets/s, Flow Bytes/s

#### 10. **Down/Up Ratio**

#### 11. **Average Packet Size**

#### 12. **Subflow:**

- Fwd/Bwd Packets, Bytes

#### 13. **Init\_Win\_bytes (TCP Window)**

#### 14. **Active/Idle Times:**

- Active Mean, Std, Max, Min
- Idle Mean, Std, Max, Min

### 1.5.4 3.4 Preprocessing Steps

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler

def preprocess_cicids2017(file_path):
 """
 Preprocess CICIDS2017 dataset
 """

 # Load data
 df = pd.read_csv(file_path)

 # Handle missing values and infinities
 df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

```

df.fillna(0, inplace=True)

Remove duplicates
df.drop_duplicates(inplace=True)

Map labels
label_mapping = {
 'BENIGN': 'Normal',
 'Bot': 'Botnet',
 'PortScan': 'PortScan',
 'DDoS': 'DDoS',
 'DoS GoldenEye': 'DoS',
 'DoS Hulk': 'DoS',
 'DoS Slowhttptest': 'DoS',
 'DoS slowloris': 'DoS',
 'FTP-Patator': 'BruteForce',
 'SSH-Patator': 'BruteForce',
 'Web Attack – Brute Force': 'WebAttack',
 'Web Attack – XSS': 'WebAttack',
 'Web Attack – Sql Injection': 'WebAttack',
 'Infiltration': 'Infiltration',
 'Heartbleed': 'Heartbleed'
}

df['Label'] = df['Label'].str.strip().map(label_mapping)

Select features (exclude IDs, IPs, timestamps)
feature_cols = [col for col in df.columns if col not in
 ['Flow ID', 'Source IP', 'Source Port', 'Destination IP',
 'Destination Port', 'Timestamp', 'Label']]

X = df[feature_cols]
y = df['Label']

Normalize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

return X_scaled, y, scaler

```

### ### 3.5 Usage Guidelines

**\*\*Recommended Split:\*\***

- Use Monday + Tuesday (first 2 days) **for** training
- Wednesday-Thursday **for** validation
- Friday **for** testing
- This ensures temporal separation **and** realistic scenario



**\*\*Class Distribution:\*\***

- Normal: ~83%
- DoS: ~11%
- PortScan: ~3%
- Other attacks: <3%

---

## ## 4. UNSW-NB15 Dataset

### ### 4.1 Overview

The UNSW-NB15 dataset was created by UNSW Canberra using IXIA traffic generator to simulate

**\*\*Statistics:\*\***

- Total records: 2,540,044
- Training set: 175,341
- Test set: 82,332
- Features: 49 (42 numerical, 7 categorical)
- Attack categories: 9

### ### 4.2 Attack Categories

Category	Description	% in Dataset
Normal	Legitimate traffic	56%
Generic	Attack affecting block cipher	18%
Exploits	Dictionary brute force, backdoor	11%
Fuzzers	Fuzzing tools	8%
DoS	Denial of Service	4%
Reconnaissance	Scanning, probing	3%
Analysis	Port scanning, spam	<1%
Backdoor	Bypassing authentication	<1%
Shellcode	Code injection	<1%
Worms	Self-replicating malware	<1%

### ### 4.3 Feature Categories

**\*\*Flow Features:\*\***

- srcip, sport, dstip, dsport, proto (TCP/UDP/ICMP)
- state (connection state)
- dur (record duration)
- sbytes, dbytes (source/destination bytes)
- sttl, dttl (source/destination time to live)
- sloss, dloss (source/destination packet loss)

**\*\*Content Features:\*\***

- sload, dload (source/destination bits per second)
- spkts, dpkts (source/destination packet count)
- swin, dwin (source/destination TCP window size)
- stcpb, dtcpb (TCP base sequence number)
- smeansz, dmeansz (mean flow packet size)

**\*\*Time Features:\*\***

- sintpkt, dintpkt (inter-packet arrival time)
- sjit, djit (jitter)
- sinpkt, dinpkt (inter-arrival time)

**\*\*Additional Features:\*\***

- tcprtt (TCP round-trip time)
- synack, ackdat (TCP connection setup times)
- ct\_state\_ttl, ct\_flw\_http\_mthd, is\_ftp\_login, etc.

### ### 4.4 Preprocessing

```
```python
```

```
def preprocess_unsw_nb15(train_path, test_path):
```

```
    """
```

```
    Preprocess UNSW-NB15 dataset
```

```
    """
```

```
    # Column names
```

```
    columns = ['srcip', 'sport', 'dstip', 'dsport', 'proto', 'state', 'dur',
               'sbytes', 'dbytes', 'sttl', 'dttl', 'sloss', 'dloss', 'service',
               'sload', 'dload', 'spkts', 'dpkts', 'swin', 'dwin', 'stcpb',
               'dtcpb', 'smeansz', 'dmeansz', 'trans_depth', 'res_bdy_len',
               'sjit', 'djit', 'stime', 'ltime', 'sintpkt', 'dintpkt',
               'tcprtt', 'synack', 'ackdat', 'is_sm_ips_ports', 'ct_state_ttl',
               'ct_flw_http_mthd', 'is_ftp_login', 'ct_ftp_cmd', 'ct_srv_src',
               'ct_srv_dst', 'ct_dst_ltm', 'ct_src_ltm', 'ct_src_dport_ltm',
               'ct_dst_sport_ltm', 'ct_dst_src_ltm', 'attack_cat', 'label']
```

```
    # Load
```

```
    train_df = pd.read_csv(train_path, names=columns, skiprows=1)
```

```
    test_df = pd.read_csv(test_path, names=columns, skiprows=1)
```

```
    # Handle missing and infinite values
```

```
    for df in [train_df, test_df]:
```

```
        df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

```
        df.fillna(0, inplace=True)
```

```
    # Encode categorical features
```

```
    categorical_cols = ['proto', 'state', 'service']
```

```
    for col in categorical_cols:
```

```

le = LabelEncoder()
combined = pd.concat([train_df[col], test_df[col]])
le.fit(combined)
train_df[col] = le.transform(train_df[col])
test_df[col] = le.transform(test_df[col])

# Select features (exclude IPs, timestamps)
feature_cols = [col for col in columns if col not in
                 ['srcip', 'dstip', 'stime', 'ltime', 'attack_cat', 'label']]

X_train = train_df[feature_cols]
y_train = train_df['attack_cat']
X_test = test_df[feature_cols]
y_test = test_df['attack_cat']

# Normalize
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

return X_train_scaled, y_train, X_test_scaled, y_test, scaler

```

1.6 5. IoT-23 Dataset

1.6.1 5.1 Overview

The IoT-23 dataset contains real IoT botnet traffic captured from 23 infected IoT devices.

Statistics: - Total captures: 23 scenarios - Total packets: 325 million+ - Duration: 20+ scenarios, each 1-4 hours - Malware: Mirai, Torii, Gafgyt, etc.

1.6.2 5.2 Malware Types

Mirai Variants: - Most common IoT botnet - DDoS attacks, brute force scanning - Multiple variants captured

Torii: - Sophisticated, persistent botnet - Multiple exploitation methods - Telnet brute force, command injection

Others: - Gafgyt, Kenjiro, Hide and Seek, Hakai

1.6.3 5.3 Feature Extraction

Since IoT-23 provides PCAP files, features must be extracted:

```

from scapy.all import rdpcap, IP, TCP, UDP
import pandas as pd

```

```

def extract_features_iot23(pcap_file):
    """
    Extract features from IoT-23 PCAP files
    """

    packets = rdpcap(pcap_file)
    features = []

    for pkt in packets:
        if IP in pkt:
            feature = {
                'timestamp': float(pkt.time),
                'src_ip': pkt[IP].src,
                'dst_ip': pkt[IP].dst,
                'protocol': pkt[IP].proto,
                'packet_len': len(pkt),
                'ttl': pkt[IP].ttl,
                'flags': pkt[IP].flags
            }

            if TCP in pkt:
                feature.update({
                    'src_port': pkt[TCP].sport,
                    'dst_port': pkt[TCP].dport,
                    'tcp_flags': pkt[TCP].flags,
                    'window_size': pkt[TCP].window
                })
            elif UDP in pkt:
                feature.update({
                    'src_port': pkt[UDP].sport,
                    'dst_port': pkt[UDP].dport
                })

            features.append(feature)

    df = pd.DataFrame(features)
    return df

```

1.7 6. 5G/Custom Dataset Collection

1.7.1 6.1 Collection Plan

Testbed Setup: - Open5GS or free5GC core network - srsRAN or OAI for RAN simulation - UERANSIM for UE simulation - Mininet for network topology

Attack Scenarios: - Network slicing isolation violations - Signaling storm attacks - Authentication and key agreement (AKA) attacks - Denial of service on core functions

(AMF, SMF) - Man-in-the-middle on user plane

1.7.2 6.2 Data Collection Tools

```
import pyshark
import pandas as pd

def capture_5g_traffic(interface='eth0', duration=3600):
    """
    Capture 5G network traffic
    """

    capture = pyshark.LiveCapture(interface=interface)
    features = []

    for packet in capture.sniff_continuously(packet_count=100000):
        try:
            feature = {
                'timestamp': packet.sniff_timestamp,
                'src_ip': packet.ip.src,
                'dst_ip': packet.ip.dst,
                'protocol': packet.highest_layer,
                'length': packet.length
            }

            if hasattr(packet, 'tcp'):
                feature['src_port'] = packet.tcp.srcport
                feature['dst_port'] = packet.tcp.dstport

            features.append(feature)
        except AttributeError:
            continue

    df = pd.DataFrame(features)
    return df
```

1.8 7. Data Preprocessing Pipeline

1.8.1 7.1 Complete Pipeline

```
class DataPreprocessor:
    def __init__(self):
        self.scaler = StandardScaler()
        self.label_encoders = {}
        self.feature_selector = None

    def fit_transform(self, X, y):
```

```

    """
    Fit and transform training data
    """
    X_processed = X.copy()

    # 1. Handle missing values
    X_processed = self._handle_missing(X_processed)

    # 2. Remove outliers
    X_processed = self._remove_outliers(X_processed)

    # 3. Encode categorical features
    X_processed = self._encode_categorical(X_processed, fit=True)

    # 4. Feature engineering
    X_processed = self._engineer_features(X_processed)

    # 5. Feature selection
    X_processed = self._select_features(X_processed, y, fit=True)

    # 6. Normalize
    X_processed = self.scaler.fit_transform(X_processed)

    # 7. Handle class imbalance
    X_balanced, y_balanced = self._balance_classes(X_processed, y)

    return X_balanced, y_balanced

def transform(self, X):
    """
    Transform test data
    """
    X_processed = X.copy()
    X_processed = self._handle_missing(X_processed)
    X_processed = self._encode_categorical(X_processed, fit=False)
    X_processed = self._engineer_features(X_processed)
    X_processed = self._select_features(X_processed, fit=False)
    X_processed = self.scaler.transform(X_processed)
    return X_processed

def _handle_missing(self, X):
    """Handle missing values"""
    X.fillna(X.median(), inplace=True)
    return X

def _remove_outliers(self, X, threshold=3):
    """Remove outliers using Z-score"""
    z_scores = np.abs((X - X.mean()) / X.std())

```

```

X_cleaned = X[(z_scores < threshold).all(axis=1)]
return X_cleaned

def _encode_categorical(self, X, fit=False):
    """Encode categorical features"""
    categorical_cols = X.select_dtypes(include=['object']).columns

    for col in categorical_cols:
        if fit:
            le = LabelEncoder()
            X[col] = le.fit_transform(X[col].astype(str))
            self.label_encoders[col] = le
        else:
            X[col] = self.label_encoders[col].transform(X[col].astype(str))

    return X

def _engineer_features(self, X):
    """Create additional features"""
    # Example: ratios, interactions
    if 'src_bytes' in X.columns and 'dst_bytes' in X.columns:
        X['byte_ratio'] = X['src_bytes'] / (X['dst_bytes'] + 1)
    return X

def _select_features(self, X, y=None, fit=False, k=50):
    """Select top k features"""
    if fit:
        from sklearn.feature_selection import SelectKBest, f_classif
        self.feature_selector = SelectKBest(f_classif, k=k)
        X_selected = self.feature_selector.fit_transform(X, y)
    else:
        X_selected = self.feature_selector.transform(X)

    return X_selected

def _balance_classes(self, X, y):
    """Balance classes using SMOTE"""
    from imblearn.over_sampling import SMOTE

    smote = SMOTE(random_state=42)
    X_resampled, y_resampled = smote.fit_resample(X, y)

    return X_resampled, y_resampled

```

1.9 8. Train/Validation/Test Splits

1.9.1 8.1 Split Strategy

```
from sklearn.model_selection import train_test_split

def create_splits(X, y, test_size=0.15, val_size=0.15, random_state=42):
    """
    Create train/validation/test splits

    Ratios: 70% train, 15% validation, 15% test
    """

    # First split: separate test set
    X_temp, X_test, y_temp, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, stratify=y
    )

    # Second split: separate validation from train
    val_ratio = val_size / (1 - test_size)
    X_train, X_val, y_train, y_val = train_test_split(
        X_temp, y_temp, test_size=val_ratio, random_state=random_state, stratify=y_temp
    )

    print(f"Train: {len(X_train)} samples ({len(X_train)/len(X)*100:.1f}%)")
    print(f"Validation: {len(X_val)} samples ({len(X_val)/len(X)*100:.1f}%)")
    print(f"Test: {len(X_test)} samples ({len(X_test)/len(X)*100:.1f}%)")

    return X_train, X_val, X_test, y_train, y_val, y_test
```

1.10 9. Class Imbalance Handling

1.10.1 9.1 SMOTE (Synthetic Minority Over-sampling)

```
from imblearn.over_sampling import SMOTE, ADASYN
from imblearn.under_sampling import RandomUnderSampler
from imblearn.pipeline import Pipeline

# SMOTE
smote = SMOTE(sampling_strategy='minority', k_neighbors=5, random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)

# ADASYN (Adaptive Synthetic)
adasyn = ADASYN(sampling_strategy='minority', n_neighbors=5, random_state=42)
X_resampled, y_resampled = adasyn.fit_resample(X_train, y_train)

# Combined over-sampling and under-sampling
```



```

pipeline = Pipeline([
    ('over', SMOTE(sampling_strategy=0.5)),
    ('under', RandomUnderSampler(sampling_strategy=0.8))
])
X_resampled, y_resampled = pipeline.fit_resample(X_train, y_train)

```

1.10.2 9.2 Class Weights

```

from sklearn.utils.class_weight import compute_class_weight

# Compute class weights
class_weights = compute_class_weight('balanced',
                                     classes=np.unique(y_train),
                                     y=y_train)

class_weight_dict = dict(enumerate(class_weights))

# Use in model training
model.fit(X_train, y_train, class_weight=class_weight_dict)

```

1.11 10. Data Augmentation Strategies

1.11.1 10.1 Gaussian Noise Injection

```

def augment_with_noise(X, noise_level=0.01):
    """
    Add Gaussian noise to features
    """
    noise = np.random.normal(0, noise_level, X.shape)
    X_augmented = X + noise
    return X_augmented

```

1.11.2 10.2 Feature Masking

```

def augment_with_masking(X, mask_prob=0.1):
    """
    Randomly mask features (set to 0)
    """
    mask = np.random.binomial(1, 1 - mask_prob, X.shape)
    X_masked = X * mask
    return X_masked

```

1.11.3 10.3 Time-Series Augmentation

```

def augment_time_series(X, methods=['jitter', 'scaling', 'rotation']):
    """
    Augment time-series data

```

```

"""
augmented = []

for method in methods:
    if method == 'jitter':
        # Add small random noise
        X_aug = X + np.random.normal(0, 0.03, X.shape)
    elif method == 'scaling':
        # Scale by random factor
        scale = np.random.normal(1.0, 0.1)
        X_aug = X * scale
    elif method == 'rotation':
        # Rotate features (circular shift)
        shift = np.random.randint(1, X.shape[1])
        X_aug = np.roll(X, shift, axis=1)

    augmented.append(X_aug)

return np.vstack([X] + augmented)

```

1.12 Conclusion

This document provides comprehensive information about datasets used in the IDS research, including detailed specifications, preprocessing pipelines, and best practices for data preparation. The combination of established benchmarks (NSL-KDD, CICIDS2017, UNSW-NB15, IoT-23) and custom 5G data collection ensures thorough evaluation across diverse attack scenarios and network environments.

Key Points: - Multiple datasets for cross-validation and generalization - Comprehensive preprocessing pipeline for data quality - Stratified splits maintain class distribution - SMOTE and class weights address imbalance - Data augmentation improves model robustness

Next: Refer to `evaluation_metrics.md` for performance assessment methodology and `algorithms.md` for model implementation details.